Uvod u softversko inženjerstvo
Avalonia UI framework

Fakultet tehničkih nauka
Univerzitet u Novom Sadu

Kako koristiti materijal

Materijal je organizovan kao mini-projekti (po temi). Za svaku temu napravite novi projekat (Avalonia .NET App), pa onda copy/paste kod iz odgovarajuće sekcije.

Preporuka: koristite isto ime projekta (`GetStartedApp`) u svim mini-projektima, da ne biste menjali `x:Class` i namespace u snippeti-ma.

- **01-xaml** – osnove XAML-a i kontrole
- **02-events** – događaji i event handler-i
- **03-layouts** – paneli i raspored
- **04-binding** – data binding, resursi, konverzija

Kreiranje mini-projekta (CLI primer):

```
dotnet new install Avalonia.Templates
dotnet new avalonia.app -o 03-layouts -n GetStartedApp
```

Contents

1 Uvod u Avalonia **3**
1.1 Kreiranje Avalonia projekta (Rider) . . . 3
1.2 Markup i code-behind . . . 4

2 Markup (XAML) **5**

3 Code-behind **6**

4 Događaji i obrada događaja (eng. Events and handling events) **7**
4.1 Dodavanje event handler-a u Avalonia (Rider) . . . 7

5 Paneli (eng. Panels) **8**
5.1 Wrap panel . . . 8
5.2 Stack panel . . . 9
5.3 Grid panel . . . 9
5.4 Dock panel . . . 10
5.5 Primer kombinacije više panela – Complex Layout . . . 11

6 Data binding **11**
6.1 Priprema: DataContext (primer) 12
6.2 Binding na resurs – string (1) 13
6.3 Binding na resurs – boja (2) 14
6.4 One way – ka interfejsu (7) 14
6.5 Two way – bidirekciono (6) 14
6.6 One way to source – ka kodu (8) 15
6.7 Binding kontrole na kontrolu (3) 15
6.8 Binding kontrole na kontrolu - real time (4) 15
6.9 Binding kontrole na kontrolu - real time + konverzija (5) 15
6.10 Binding na listu (9) 16

1 Uvod u Avalonia

Avalonia UI je cross-platform GUI (Graphical User Interface) framework za kreiranje desktop aplikacija na Windows-u, Linux-u i macOS-u.

GUI framework omogućava da kreiramo aplikacije uz pomoć velikog opsega GUI elemenata kao što su labele (eng. labels) i polja za unos teksta (eng. textboxes). Bez GUI framework-a, morali bi da crtamo ove elemente ručno i da upravljamo sa svim korisničkim interakcijama, kao što je unos teksta ili pomeranje miša.

Avalonia sadrži set application-development alata (eng. features): XAML, kontrole, data binding, layouts, 2D grafika, animacije, stilovi, media i slično.

1.1 Kreiranje Avalonia projekta (Rider)

Preduslovi:

- .NET SDK (10.0 LTS)
- JetBrains Rider
- (Opcionalno) Rider plugin *AvaloniaRider* za live XAML preview

Instalacija Avalonia templates:

```
dotnet new list
dotnet new install Avalonia.Templates
```

Kreiranje projekta u Rider-u:

1. Na početnom ekranu izaberite **New Solution**.
2. U sekciji **Custom Templates** izaberite **Avalonia .NET App**.
3. Unesite ime rešenja/projekta i lokaciju.
4. Kliknite **Create**.

Placeholder for figure file
`assets/slika-1.png`

Figure 1: Slika 1. Kreiranje novog Avalonia projekta u Rider-u

Alternativa (command line):

```
dotnet new avalonia.app -o 01-xaml -n GetStartedApp
```

Pokretanje projekta:
U Rider-u kliknite **Run**. Alternativno, u terminalu pokrenite `dotnet run` u folderu projekta.

Placeholder for figure file
`assets/slika-2.png`

Figure 2: Slika 2. Pokretanje Avalonia aplikacije (podrazumevani ekran)

1.2 Markup i code-behind

Avalonia omogućava da se razvija aplikacija koristeći markup i code-behind.

XAML markup se obično koristi za implementaciju izgleda aplikacije dok se code-behind koristi za implementaciju ponašanja tih komponenti.

Ovo razdvajanje na izgled i ponašanje povlači neke od prednosti kao što su:

1. Cena razvoja i održavanja aplikacije je smanjena jer je znatno lakše upravljati sa aplikacijom kod koje izgled i ponašanje nisu usko vezani (eng. tightly coupled).
2. Razvoj aplikacije je ubrzan i efikasan jer dizajneri mogu implementirati izgled aplikacije nezavisno i paralelno od programera koji implementiraju ponašanje aplikacije.
3. Globalizacija i lokalizacija za desktop aplikacije je pojednostavljena.

Na slici je prikazan prazan Avalonia projekat otvoren u Rider-u. Tipični segmenti rada su:

1. **Solution** prozor (pregled projekta i fajlova).
2. **XAML editor** (npr. `MainWindow.axaml`).
3. (Opcionalno) **XAML preview** uz Rider plugin *AvaloniaRider*.
4. **Code-behind** fajl (npr. `MainWindow.axaml.cs`).

Za razliku od workflow-a sa drag&drop dizajnerom, u Rider-u se kontrole najčešće dodaju pisanjem XAML-a uz code completion i snippet-e.

Placeholder for figure file
`assets/slika-3.png`

Figure 3: Slika 3. Rider okruženje: Solution, XAML editor i (opciono) XAML preview

Otvaranjem `MainWindow.axaml.cs` dobija se code-behind.

Prelazak između XAML fajla i code-behind fajla vrši se iz **Solution** prozora ili korišćenjem pretrage (Search Everywhere).

Ako koristite XAML preview, obično je dostupan kao kartica **Preview** pored XAML editora (uz instaliran *AvaloniaRider* plugin).

Placeholder for figure file
`assets/slika-4.png`

Figure 4: Slika 4. Code-behind (`*.axaml.cs`)

Placeholder for figure file
`assets/slika-5.png`

Figure 5: Slika 5. Navigacija: otvaranje `MainWindow.axaml.cs` iz Solution prozora

2 Markup (XAML)

XAML je XML-bazirani markup jezik koji omogućava implementiranje izgleda aplikacije (eng. application's appearance). Koristi se za kreiranje prozora, jednostavnih dijaloga i kontrola a navedeno dalje možemo popunjavati sa novim kontrolama, oblicima i grafičkim elementima.

U Avalonia projektima XAML fajlovi obično imaju ekstenziju `.axaml` (npr. `MainWindow.axaml`).

Primer prikazuje Avalonia XAML kod jednostavnog prozora (tag `<Window>`) koji sadrži samo jedno dugme (tag `<Button>`).

```
<Window xmlns="https://github.com/avaloniaui"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        x:Class="GetStartedApp.MainWindow"
        Title="MainWindow" Width="800" Height="450">
  <Button Content="Click me" />
</Window>
```

Svaki element može da sadrži određene atribute, slično kao u XML-u. U primeru se vidi da tag `<Window>` sadrži Title atribut koji je zapravo tekst u title baru.

U primeru je prikazano kako se uz pomoć FontWeight property može postaviti boldovan tekst sa sadržajem "A button".

```
<Button Content="A button" FontWeight="Bold" />
```

Još jedan način definisanja atributa je prikazan. Ovaj način prikazuje atribute kao child tagove glavnog taga, a u ovom slučaju to je `<Button>`.

```
<Button>
  <Button.Content>A button</Button.Content>
  <Button.FontWeight>Bold</Button.FontWeight>
</Button>
```

Placeholder for figure file
`assets/slika-6.png`

Figure 6: Slika 6. XAML preview u Rider-u (opciono)

S obzirom da je XAML XML-bazirani jezik, UI koji se kreira predstavlja hijerarhiju ugnježdenih elementa poznatih pod nazivom element tree.

Dat je primer gde je u okviru jednog dugmeta kreirano više tekst blokova koristeći više `<TextBlock>` tagova u `<Button>` tagu.

Tekst u tekst blokovima je obojen različitim bojama koristeći atribut Foreground.

```
<Button>
  <Button.Content>
    <WrapPanel>
      <TextBlock Text="A" Foreground="Red" />
      <TextBlock Text=" " />
      <TextBlock Text="multi" Foreground="Green" />
      <TextBlock Text=" " />
      <TextBlock Text="color" Foreground="Blue" />
      <TextBlock Text=" " />
      <TextBlock Text="button" Foreground="DarkOrange" />
    </WrapPanel>
  </Button.Content>
</Button>
```

Content property dozvoljava samo jedan child element pa je bilo potrebno obuhvatiti child komeponente u WrapPanel. Paneli, kao što je WrapPanel, igraju veoma važnu ulogu u Avalonia i predstavljaju kontejnere za druge kontrole.

Isti rezultat se može postići na sledeći način. Ovaj način je jednostavniji i češće je u upotrebi.

```
<Button>
  <WrapPanel>
    <TextBlock Text="A" Foreground="Red" />
    <TextBlock Text=" " />
    <TextBlock Text="multi" Foreground="Green" />
    <TextBlock Text=" " />
    <TextBlock Text="color" Foreground="Blue" />
    <TextBlock Text=" " />
    <TextBlock Text="button" Foreground="DarkOrange" />
  </WrapPanel>
</Button>
```

3 Code-behind

Osnovna odlika svake aplikacije je da implementira funkcionalnosti, uključujući obradu događaja (eng. handling events) kao što je korisnički klik. U Avalonia, ponašanje nakon okidanja događaja se može implementirati u kodu koji je povezan sa markup-om (XAML). Ovaj tip koda se naziva code-behind.

Primer prikazuje XAML kod sa event handler-om, a zatim code-behind koji odgovara tom kodu.

```
<!-- MainWindow.axaml -->
<Button Content="Klikni me" Click="Button_Click" />

// MainWindow.axaml.cs
using Avalonia.Controls;
using Avalonia.Interactivity;

public partial class MainWindow : Window
{
    public MainWindow()
    {
        InitializeComponent();
    }

    private void Button_Click(object? sender, RoutedEventArgs e)
    {
        Title = "Kliknuto!";
    }
}
```

InitializeComponent metoda se poziva iz code-behind-a, iz konstruktora klase. Ova metoda zadužena je da spoji (eng. merge) UI koji se definiše u markup kodu sa code-behind klasom.

Napomena: InitializeComponent se generiše za vas kada se kreira aplikacija i ne treba je implementirati ručno.

4 Događaji i obrada događaja (eng. Events and handling events)

Većina UI framework-a su event driven. Sve kontrole, uključujući `Window` kontrolu, izlažu događaje na koje se može odreagovati. Možete se prijaviti (eng. subscribe) na ove događaje, što znači da će vaša aplikacija biti obaveštena kada se ovaj događaj desi i tada je moguće odreagovati na određeni način. Reakcija na događaj se zapravo definiše u event handler-u.

Postoji više tipova događaja ali oni koji se najčešće koriste su vezani za interakciju sa mišem ili tastaturom. U Avalonia se često koriste događaji kao što su: `KeyDown`, `KeyUp`, `PointerPressed`, `PointerReleased`, `PointerEntered`, `PointerExited`.

4.1 Dodavanje event handler-a u Avalonia (Rider)

1) Direktno u XAML-u

U XAML-u dodajte handler kao atribut kontrole (npr. dugmeta). Ako metoda ne postoji, Rider obično nudi quick-fix (**Alt+Enter**) da se kreira metoda u odgovarajućem code-behind fajlu.

```
<!-- MainWindow.axaml -->
<Button Content="Klik" Click="Button_Click" />

// MainWindow.axaml.cs
using Avalonia.Interactivity;

private void Button_Click(object? sender, RoutedEventArgs e)
{
    // reakcija na klik
}
```

2) Pretplata u code-behind-u

Drugi način je da se pretplatite na događaj u konstruktoru (posle `InitializeComponent()`). U tom slučaju najčešće je potrebno pronaći kontrolu po imenu (`Name` / `x:Name`).

```
// MainWindow.axaml
<Button x:Name="MyButton" Content="Klik" />
```

Placeholder for figure file
`assets/slika-14.png`

Figure 7: Slika 14. Dodavanje događaja kao XAML atribut (Rider)

```
// MainWindow.axaml.cs
using Avalonia.Controls;
using Avalonia.Interactivity;

public MainWindow()
{
    InitializeComponent();
    var button = this.FindControl<Button>("MyButton");
    button.Click += Button_Click;
}
```

Placeholder for figure file
`assets/slika-15.png`

Figure 8: Slika 15. Pretplata na događaj u code-behind-u (Rider)

Napomena: U praksi se handler-i najčešće dodaju kroz XAML ili kod; nije potrebno oslanjati se na poseban "Properties" panel.

5 Paneli (eng. Panels)

Panel, kao jedna od najvažnijih kontrola, predstavlja kontejner za druge kontrole i upravlja sa rasporedom istih na prozoru. S obzirom da Window kontrola može da ima samo jednu child kontrolu, obično se koristi panel da podeli prostor na regije gde svaka regija može ponovo biti panel.

U nastavku će biti navedeni i objašnjeni neki od panela koji se koriste u Avalonia.

5.1 Wrap panel

WrapPanel pozicionira svaku od svojih child kontrola jednu do druge, horizontalno (default varijanta) ili vertikalno, dokle god ima dovoljno prostora. Kada ne postoji dovoljno prostora, child elementi se wrap-uju u novi red i nastavlja se njihovo ređanje.

WrapPanel treba koristiti kada imamo listu kontrola koje želimo da rasporedimo vertikalno ili horizontalno.

Kada WrapPanel koristi horizontalnu orijentaciju, child kontrole će imati istu visinu, zasnovanu na visini najvišeg elementa. Kada je WrapPanel vertikalno orijentisan, child kontrole će imati istu širinu kao najširi element.

Primer XAML koda (Avalonia) za WrapPanel:

```
<!-- MainWindow.axaml -->
<Window xmlns="https://github.com/avaloniaui"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        x:Class="GetStartedApp.MainWindow"
        Title="WrapPanel primer" Width="520" Height="260">
  <WrapPanel Margin="12">
    <Button Content="JEDAN" Width="120" Height="40" Margin="6" />
    <Button Content="DVA" Width="120" Height="40" Margin="6" />
    <Button Content="TRI" Width="120" Height="40" Margin="6" />
    <Button Content="CETIRI" Width="120" Height="40" Margin="6" />
    <Button Content="PET" Width="120" Height="40" Margin="6" />
    <Button Content="SEST" Width="120" Height="40" Margin="6" />
    <Button Content="SEDAM" Width="120" Height="40" Margin="6" />
  </WrapPanel>
</Window>
```

Očekivano: dugmad se ređaju levo–desno i "prelamaju" u novi red kada nema dovoljno širine.

5.2 Stack panel

StackPanel je veoma sličan Wrap panelu ali sa jednom važnom razlikom, ne wrap-uje sadržaj. Omogućava da se elementi ređaju jedan na drugi, u određenom smeru.

Primer XAML koda (Avalonia) za StackPanel:

```
<!-- MainWindow.axaml -->
<Window xmlns="https://github.com/avaloniaui"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        x:Class="GetStartedApp.MainWindow"
        Title="StackPanel primer" Width="420" Height="260">
  <StackPanel Margin="12" Spacing="8">
    <Button Content="JEDAN" Height="36" />
    <Button Content="DVA" Height="36" />
    <Button Content="TRI" Height="36" HorizontalAlignment="Left" Width="160" />
    <Button Content="CETIRI" Height="36" />
  </StackPanel>
</Window>
```

Očekivano: dugmad se ređaju u koloni (vertikalno) bez wrap-ovanja.

Treba primetiti da je default ponašanje Stack panela da razvlači (eng. Stretch) svoje chid kontrole.

StackPanel koji ima vertikalnu orijentaciju, sve svoje child kontrole rasteže horizontalno. Kada je u pitanju StackPanel sa horizontalnom orijentacijom, sve child kontrole se rastežu vertikalno.

StackPanel omogućava to zato što su HorizontalAlignment ili VerticalAlignment property na njegovim child kontrolama postavljeni na Stretch. Ovo ponašanje se može promeniti.

Takođe treba uočiti da je default orijentacija za StackPanel Vertical, za razliku od WrapPanel kod kog je Horizontal.

5.3 Grid panel

Grid može da sadrži više redova i kolona. Moguće je definisati visinu za svaki od redova ili širinu za svaku kolonu. Visina ili širina se mogu definisati na više načina, kao broj piksela, procentima u zavisnosti od slobodnog prostora ili da se vrednost prilagodi veličini sadržaja.

Uz pomoć RowDefinition atributa odredili smo da će Grid panel imati 3 reda. Za prva dva reda visina je postavljena na Auto.

Kako smo odredili koliko će Grid imati redova, možemo da odredimo i koliko kolona želimo u Grid-u. Ovo ćemo uraditi uz pomoć ColumnDefinition atributa. Za prve dve kolone širina je postavljena na Auto dok je za treću uzeta default vrednost.

Default vrednost za širinu je GridLength i predstavlja "1*" veličinu.

Primer XAML koda (Avalonia) za Grid:

```
<!-- MainWindow.axaml -->
<Window xmlns="https://github.com/avaloniaui"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        x:Class="GetStartedApp.MainWindow"
        Title="Grid primer" Width="520" Height="320">
  <Grid Margin="12" RowDefinitions="Auto,Auto,*" ColumnDefinitions="Auto,Auto,*">
    <Button Content="JEDAN" Grid.Row="0" Grid.Column="0" Background="LightSalmon"
    ↪  Margin="4" />
    <Button Content="DVA" Grid.Row="0" Grid.Column="1" Background="LightSkyBlue"
    ↪  Margin="4" />
    <Button Content="TRI" Grid.Row="0" Grid.Column="2" Background="LightGreen" Margin="4"
    ↪  />

    <Button Content="CETIRI" Grid.Row="1" Grid.Column="0" Grid.RowSpan="2"
    ↪  Background="Khaki" Margin="4" />
    <Button Content="PET" Grid.Row="1" Grid.Column="1" Grid.ColumnSpan="2"
    ↪  Background="Plum" Margin="4" />

    <Button Content="SEST" Grid.Row="2" Grid.Column="1" Grid.ColumnSpan="2"
    ↪  Background="Gainsboro" Margin="4" />
  </Grid>
</Window>
```

U XAML kodu možemo primetiti da je dodato 6 dugmića i da je za svaki definisano u kojoj koloni i redu se nalazi. Ovakvo ponašanje omogućavaju Grid.Column i Grid.Row atributi.

Osim toga, dodati su atributi koji određuju boju pozadine, kao i ColumnSpan i RowSpan.

Očekivano: kontrole se raspoređuju po redovima/kolonama, a span-ovi proširuju kontrolu preko više ćelija.

5.4 Dock panel

DockPanel olakšava raspoređivanje sadržaja u 4 smera (top, bottom, left, right). Predstavlja najbolji izbor kada je potrebno podeliti prozor u specifične sekcije, naročito zbog toga što poslednji element u DockPanelu, po default-u, zauzima preostali prostor (center). Na taj način se sav prostor u prozoru zauzima.

DockPanel.Dock property omogućava da odredimo smer gde će kontrola biti postavljena.

Primer XAML koda (Avalonia) za DockPanel:

```
<!-- MainWindow.axaml -->
<Window xmlns="https://github.com/avaloniaui"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        x:Class="GetStartedApp.MainWindow"
        Title="DockPanel primer" Width="520" Height="320">
  <DockPanel Margin="12" LastChildFill="True">
    <Button Content="JEDAN" DockPanel.Dock="Top" Height="40" Margin="4" />
    <Button Content="DVA" DockPanel.Dock="Left" Width="120" Margin="4" />
    <Button Content="TRI" DockPanel.Dock="Bottom" Height="40" Margin="4" />
    <Button Content="CETIRI" Margin="4" />
  </DockPanel>
</Window>
```

Dugmići u drugom panelu su raspoređeni u različitim smerovima a dugme 'CETIRI' osim toga zauzima i centralni deo jer je poslednji naveden.

Očekivano: poslednji element popunjava preostali prostor (center) ako je `LastChildFill=True`.

5.5 Primer kombinacije više panela – Complex Layout

U primeru je iskorišćen DockPanel da bi se prostor podelio na sekcije.

Na vrhu (DockPanel.Dock = "Top") postavljen je jedan TextBlock. Ista stvar urađena je i na dnu prozora.

Ostatak prostora popunjen je sa novim DockPanelom u okviru kog je kreiran StackPanel i dodat jedan TextBlock. StackPanel u sebi objedinjuje dva dugmeta a svaki od njih određen je atributima kao što su Height, Width i Margin.

Ovim primerom pokazano je kako se kompozicijom više panela može napraviti željeni raspored komponenti.

Primer XAML koda (Avalonia) za kombinaciju više panela (complex layout):

```
<!-- MainWindow.axaml -->
<Window xmlns="https://github.com/avaloniaui"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        x:Class="GetStartedApp.MainWindow"
        Title="Complex layout" Width="720" Height="420">
  <DockPanel>
    <TextBlock DockPanel.Dock="Top" Text="Gornji tekst" Margin="12" FontSize="18" />
    <TextBlock DockPanel.Dock="Bottom" Text="Donji tekst" Margin="12" />

    <DockPanel Margin="12">
      <StackPanel DockPanel.Dock="Left" Spacing="8" Margin="0,0,12,0">
        <Button Content="DUGME 1" Height="36" Width="160" Margin="0,0,0,8" />
        <Button Content="DUGME 2" Height="36" Width="160" Margin="0" />
      </StackPanel>

      <TextBlock Text="Sadržaj" VerticalAlignment="Center" />
    </DockPanel>
  </DockPanel>
</Window>
```

Očekivano: gornji i donji tekst su dock-ovani, levo su dugmad, a ostatak prostora zauzima sadržaj.

6 Data binding

Većina aplikacija omogućava korisniku da manipuliše sa nekim podacima. Kada su podaci učitani u aplikaciju (npr. iz neke baze podataka) potrebno ih je prikazati korisniku. Osim toga, korisniku se često omogućava da menja te podatke pa se mora voditi računa da bude tačno stanje podataka u bazi.

Da bi olakšala prikaz i interakciju sa podacima, Avalonia koristi data binding. Data binding predstavlja mehanizam koji omogućava komunikaciju UI elemenata sa podacima. Kada je binding uspostavljen i kada se podaci promene, moguće je automatski promeniti i UI elemente koji zavise od tih podataka. Obrnuto je takođe moguće.

Moguće je vršiti binding između dve kontrole kao i binding na resurs.

Da bi se mogle detektovati promene na source-u (primenjivo na OneWay i TwoWay binding), source mora implementirati odgovarajući mehanizam notifikacije promena, kao što je `INotifyPropertyChanged`.

Na primeru biće objašnjeni različiti načini upotrebe binding-a. U nastavku koristimo jednostavan ViewModel kao `DataContext`.

6.1 Priprema: DataContext (primer)

Napravite klasu `BindingDemoViewModel.cs` i postavite je kao `DataContext` u `MainWindow.axaml.cs`.

```
// BindingDemoViewModel.cs
using System.Collections.ObjectModel;
using System.ComponentModel;
using System.Runtime.CompilerServices;

namespace GetStartedApp;

public class BindingDemoViewModel : INotifyPropertyChanged
{
    private string _test1 = "Pocetna vrednost";
    public string Test1
    {
        get => _test1;
        set
        {
            if (_test1 == value) return;
            _test1 = value;
            OnPropertyChanged();
        }
    }

    private string _test2 = "Edituj me";
    public string Test2
    {
        get => _test2;
        set
        {
            if (_test2 == value) return;
            _test2 = value;
            OnPropertyChanged();
        }
    }

    private string _onlySource = "";
    public string OnlySource
    {
        get => _onlySource;
        set
        {
            if (_onlySource == value) return;
            _onlySource = value;
            OnPropertyChanged();
        }
    }

    private string _immediateText = "";
    public string ImmediateText
    {
        get => _immediateText;
        set
        {
            if (_immediateText == value) return;
            _immediateText = value;
            OnPropertyChanged();
        }
    }
```

```
    private string _lostFocusText = "";
    public string LostFocusText
    {
        get => _lostFocusText;
        set
        {
            if (_lostFocusText == value) return;
            _lostFocusText = value;
            OnPropertyChanged();
        }
    }

    public ObservableCollection<string> Test3 { get; } = new()
    {
        "Audi",
        "BMW",
        "Opel",
    };

    private string? _selectedTest3;
    public string? SelectedTest3
    {
        get => _selectedTest3;
        set
        {
            if (_selectedTest3 == value) return;
            _selectedTest3 = value;
            OnPropertyChanged();
        }
    }

    private int _itemCounter = 1;
    public void AddItem() => Test3.Add($"Item {_itemCounter++}");

    public event PropertyChangedEventHandler? PropertyChanged;
    private void OnPropertyChanged([CallerMemberName] string? propertyName = null) =>
        PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(propertyName));
}

// MainWindow.axaml.cs
using Avalonia.Controls;

namespace GetStartedApp;

public partial class MainWindow : Window
{
    public BindingDemoViewModel ViewModel { get; } = new();

    public MainWindow()
    {
        InitializeComponent();
        DataContext = ViewModel;
    }
}
```

6.2 Binding na resurs – string (1)

Resursi se najčešće definišu u okviru `Window.Resources` bloka i koriste se preko `StaticResource`.

```
<!-- MainWindow.axaml (primer) -->
<Window xmlns="https://github.com/avaloniaui"
```

```
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        x:Class="GetStartedApp.MainWindow"
        Title="Resources (string)" Width="520" Height="220">
  <Window.Resources>
    <x:String x:Key="strTestBinding">Hello from resource</x:String>
  </Window.Resources>

  <StackPanel Margin="12" Spacing="8">
    <TextBox Text="{StaticResource strTestBinding}" />
  </StackPanel>
</Window>
```

6.3 Binding na resurs – boja (2)

Primer resursa koji se koristi kao brush (npr. boja selekcije teksta).

```
<!-- MainWindow.axaml (primer) -->
<Window xmlns="https://github.com/avaloniaui"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        x:Class="GetStartedApp.MainWindow"
        Title="Resources (brush)" Width="520" Height="240">
  <Window.Resources>
    <SolidColorBrush x:Key="clrSpecialColor" Color="OrangeRed" />
  </Window.Resources>

  <StackPanel Margin="12" Spacing="8">
    <TextBox Text="Selektuj deo teksta" SelectionBrush="{StaticResource clrSpecialColor}"
    ↪  />
  </StackPanel>
</Window>
```

6.4 One way – ka interfejsu (7)

OneWay binding – promene na source property automatski osvežavaju target property, ali izmene na kontroli ne menjaju source.

```
<!-- unutar Window.Content -->
<StackPanel Margin="12" Spacing="8">
  <TextBlock Text="OneWay (Test1)" FontWeight="Bold" />
  <TextBox Text="{Binding Test1, Mode=OneWay}" />
  <Button Content="Promeni Test1 u kodu" Click="SetTest1_Click" />
</StackPanel>

// MainWindow.axaml.cs (dodati u klasu)
using System;
using Avalonia.Interactivity;

private void SetTest1_Click(object? sender, RoutedEventArgs e)
{
    ViewModel.Test1 = $"Test1: {DateTime.Now:HH:mm:ss}";
}
```

6.5 Two way – bidirekciono (6)

TwoWay binding – promene na source ili target property osvežavaju drugu stranu. `TextBox.Text` je u Avalonia podrazumevano `TwoWay`.

```
<!-- unutar Window.Content -->
<StackPanel Margin="12" Spacing="8">
  <TextBlock Text="TwoWay (Test2)" FontWeight="Bold" />
```

```
  <TextBox Text="{Binding Test2}" />
  <TextBlock Text="{Binding Test2, StringFormat='Trenutno: {0}'}" />
</StackPanel>
```

6.6 One way to source – ka kodu (8)

OneWayToSource binding – promene na kontroli osvežavaju source, ali promene na source-u ne osvežavaju kontrolu.

```
<!-- unutar Window.Content -->
<StackPanel Margin="12" Spacing="8">
  <TextBlock Text="OneWayToSource (OnlySource)" FontWeight="Bold" />
  <TextBox Watermark="Kucaj ovde" Text="{Binding OnlySource, Mode=OneWayToSource}" />
  <TextBlock Text="{Binding OnlySource, StringFormat='Source: {0}'}" />
</StackPanel>
```

6.7 Binding kontrole na kontrolu (3)

Osim binding-a na `DataContext`, moguće je bindovati kontrolu direktno na kontrolu. U Avalonia možete koristiti `#` sintaksu (named controls).

```
<!-- unutar Window.Content -->
<StackPanel Margin="12" Spacing="8">
  <TextBlock Text="Control-to-control binding" FontWeight="Bold" />
  <Slider x:Name="mySlider" Minimum="0" Maximum="100" Value="25" />
  <TextBlock Text="{Binding #mySlider.Value, StringFormat='Slider: {0:0}'}" />
</StackPanel>
```

6.8 Binding kontrole na kontrolu - real time (4)

`UpdateSourceTrigger` određuje kada će se promena sa kontrole preneti na source. U Avalonia, `TextBox.Text` se podrazumevano sinhronizuje na svako kucanje (`PropertyChanged`).

```
<!-- unutar Window.Content -->
<StackPanel Margin="12" Spacing="8">
  <TextBlock Text="UpdateSourceTrigger" FontWeight="Bold" />

  <TextBox Watermark="PropertyChanged (default)" Text="{Binding ImmediateText}" />
  <TextBlock Text="{Binding ImmediateText, StringFormat='Immediate: {0}'}" />

  <TextBox Watermark="LostFocus" Text="{Binding LostFocusText,
  ↪  UpdateSourceTrigger=LostFocus}" />
  <TextBlock Text="{Binding LostFocusText, StringFormat='LostFocus: {0}'}" />
</StackPanel>
```

6.9 Binding kontrole na kontrolu - real time + konverzija (5)

Ako se u procesu binding-a želi iskoristiti konverzija, koristite `Converter`. U Avalonia se kreira implementacijom `IValueConverter`.

```
// MultiplyConverter.cs
using System;
using System.Globalization;
using Avalonia.Data.Converters;

namespace GetStartedApp;

public class MultiplyConverter : IValueConverter
{
```

```
    public double Factor { get; set; } = 1;

    public object? Convert(object? value, Type targetType, object? parameter, CultureInfo
    ↪  culture)
    {
        if (value is double d)
            return (d * Factor).ToString("0.##", culture);
        return value?.ToString();
    }

    public object? ConvertBack(object? value, Type targetType, object? parameter,
    ↪  CultureInfo culture)
    {
        if (value is string s && double.TryParse(s, NumberStyles.Any, culture, out var
        ↪  d))
            return d / Factor;
        return value;
    }
}
```

```
<!-- MainWindow.axaml (primer) -->
<Window xmlns="https://github.com/avaloniaui"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        xmlns:local="using:GetStartedApp"
        x:Class="GetStartedApp.MainWindow"
        Title="Converter" Width="520" Height="260">
  <Window.Resources>
    <local:MultiplyConverter x:Key="Mul2" Factor="2" />
  </Window.Resources>

  <StackPanel Margin="12" Spacing="8">
    <Slider x:Name="convSlider" Minimum="0" Maximum="50" Value="10" />
    <TextBlock Text="{Binding #convSlider.Value, StringFormat='Original: {0:0}'}" />
    <TextBlock Text="{Binding #convSlider.Value, Converter={StaticResource Mul2},
    ↪  StringFormat='x2: {0}'}" />
  </StackPanel>
</Window>
```

6.10 Binding na listu (9)

Primer binding-a `ComboBox` kontrole na kolekciju (`ObservableCollection`).

```
<!-- unutar Window.Content -->
<StackPanel Margin="12" Spacing="8">
  <TextBlock Text="Binding na listu (Test3)" FontWeight="Bold" />

  <ComboBox ItemsSource="{Binding Test3}"
            SelectedItem="{Binding SelectedTest3}"
            Width="200" />

  <TextBlock Text="{Binding SelectedTest3, StringFormat='Selektovano: {0}'}" />

  <Button Content="Dodaj element" Click="AddItem_Click" Width="200" />
</StackPanel>
```

```
// MainWindow.axaml.cs (dodati u klasu)
using Avalonia.Interactivity;

private void AddItem_Click(object? sender, RoutedEventArgs e)
{
    ViewModel.AddItem();
}
```

DODATAK

Kontrole (Avalonia reference):

- https://docs.avaloniaui.net/docs/reference/controls/

Često korišćeni layout-i (concept):

- **Canvas:** Child controls provide their own layout.
- **DockPanel:** Child controls are aligned to the edges of the panel.
- **Grid:** Child controls are positioned by rows and columns.
- **StackPanel:** Child controls are stacked either vertically or horizontally.
- **WrapPanel:** Child controls are positioned in left-to-right order and wrapped to the next line when there are more controls on the current line than space allows.

Korisni linkovi:

- https://docs.avaloniaui.net/docs/get-started/
- https://docs.avaloniaui.net/docs/get-started/xaml-previewers
- https://docs.avaloniaui.net/docs/get-started/wpf/
- https://docs.avaloniaui.net/docs/basics/data/data-binding/
- https://docs.avaloniaui.net/docs/basics/data/data-binding/data-binding-syntax
- https://docs.avaloniaui.net/docs/guides/data-binding/binding-to-controls
- https://play.avaloniaui.net/